MPCP

Multi-Port Catch Protocol

Protocol Specification — Final Draft v0.5

2025

Abstract

MPCP (Multi-Port Catch Protocol) is a peer-to-peer file transfer protocol designed for stealth, resistance to passive network surveillance, and high performance. v0.5 refines the final draft with five targeted improvements: z-score normalised tripwire thresholds replacing the fixed millisecond value; a trimmed-mean + MAD + EWMA RTT estimation pipeline replacing the single trimmed-mean estimator; explicit deterministic CLI profile parameter sets; tightened constant-time blind key exchange specification; and a full implementation testing plan with Linux-specific guidance for SCHED_FIFO and CPU isolation.

Changelog v0.5: *Section 7: RTT pipeline replaced with trimmed-mean + MAD + EWMA. Section 21: Tripwire threshold replaced with z-score approach. Section 16: CLI profiles made deterministic with explicit parameter tables. Section 8: Blind key exchange timing tightened with precise operation sequence. Section 25 (new): Implementation and Testing Plan.*

1. Introduction

MPCP (Multi-Port Catch Protocol) is a peer-to-peer file transfer protocol that treats network transfer as a cooperative timing problem rather than a connection problem. Both endpoints independently derive identical port numbers, timing windows, and cryptographic keys from a combination of shared observable network measurements and out-of-band secrets. No persistent ports are opened, no session handshake is observable, and no key material is ever transmitted.

v0.5 is the refined final draft. It incorporates all security hardening from v0.3, all performance optimisations from v0.4, and five targeted refinements identified during implementation review: statistically robust RTT estimation, network-adaptive tripwire thresholds, deterministic CLI profiles, tightened key exchange specification, and a comprehensive testing plan.

---

2. Design Goals

Goal	Mechanism
No persistent open ports	Ports open <200ms per chunk on HKDF-derived numbers
No observable session structure	No handshake, no teardown, randomised counts, silence = completion
No transmitted key material	4-source HKDF: ping timing + PSK + OS random + timestamp
No file signature leakage	zstd + XChaCha20 + uniform padding + ghost chunks
No replay or injection	Per-session nonce, AEAD with sequence index in AD, bitmask dedup
Endpoint authentication	PSK implicit + optional Ed25519
Interception detection	Z-score timing oracle + chi-squared loss analysis + decoy ping
Silent abort on detection	Immediate sodium_memzero, local canary log, no network signal
Robust RTT estimation	Trimmed mean + MAD + EWMA pipeline, stable on WiFi and internet
Adaptive performance	Streaming pipeline, configurable pipeline depth, adaptive catch window
NAT compatibility	UDP hole punching, hashed rendezvous tokens
DPI resistance	Disguised calibration, randomised ping count, decoy encoding
Memory safety	sodium_mlock, per-chunk zero, streaming O(chunk) memory
Full configurability	Every parameter exposed; deterministic CLI profiles

---

3. Terminology

Term	Definition
PC1 (Receiver)	The machine accepting the file. Generates candidate keys.
PC2 (Sender)	The machine sending the file. Drives calibration and transfer.

PSK	Pre-Shared Key. Agreed out-of-band. Never transmitted.
Session nonce	256-bit random value generated fresh each session. Included in all HKDF derivations.
Master secret	HKDF output combining ping timing, PSK, OS random, and timestamp.
Session key	Final per-session key derived from master secret and blind-selected candidate.
Keystream	Single HKDF-Expand output sliced into per-chunk keys and port seeds.
Ghost chunk	Full-size encrypted chunk containing only random padding.
Catch window	Duration PC1 holds a derived port open awaiting a specific chunk.
MAD	Median Absolute Deviation. Robust spread estimator, resistant to outliers and skew.
EWMA	Exponentially Weighted Moving Average. Continuously updated estimate tracking current conditions.
Z-score	Number of standard deviations a sample deviates from the baseline mean. Used for network-adaptive tripwire threshold.
Tripwire	Active interception detection. Triggers silent abort on anomaly.
Canary	Local timestamp log entry written on tripwire trigger.
Decoy encoding	Optional formatting of encrypted chunks to resemble C source code.
Pipeline depth	Number of chunks in flight simultaneously.
SCHED_FIFO	Linux real-time scheduling policy for the port timing thread.
isolcpus	Linux kernel parameter to dedicate a CPU core, preventing OS scheduler use.

---

4. Protocol Phases Overview

Phase	Name	Description
0	Pre-Session Setup	PSK agreement out-of-band. Optional Ed25519. NAT rendezvous if needed.
1	Accept Mode	PC1 passively monitors for calibration traffic. Port timing thread started at SCHED_FIFO.
2	Calibration	Randomised-count ping burst or slow mode. Trimmed mean + MAD + EWMA RTT pipeline. Adaptive catch window.
3	Key Exchange	Parallel candidate key send. Constant-time blind selection with precisely specified timing. Session key derived.

4	File Transfer	Streaming pipeline. Ghost chunks on-the-fly. Pipelined ACKs. Z-score tripwire monitoring.
5	Reassembly	Streaming receive, verify, decrypt, write. Ghost discard. Per-chunk key zero. Decompress.

---

5. Phase 0 — Pre-Session Setup

5.1 Pre-Shared Key

Both users agree on a PSK through an out-of-band channel before any network activity. The PSK contributes entropy independent of all observable network traffic and provides implicit endpoint authentication. Never transmitted.

> **SECURITY:** *Weak PSK undermines multi-source entropy. MPCP enforces psk_min_entropy at startup.*

Parameter	Default	Description
psk	(required)	Passphrase via CLI or interactive prompt. Never stored unless psk_store = true.
psk_min_entropy	40	Minimum estimated entropy bits. Session refused below threshold.
psk_store	false	Store PSK in encrypted system keyring.

5.2 PSK Generator

Built-in Diceware-style generator producing ~100-bit passphrases:

```
mpcp --generate-psk
  coral-tandem-velvet-sunrise-hinge-frozen-atlas-drum
```

5.3 Optional Ed25519 Authentication

> **TOGGLE:** *AUTH_MODE — disabled by default. Enable with --auth or auth_mode = ed25519.*

Long-term Ed25519 keypairs generated on first run. Public keys exchanged out-of-band. Session transcript hash signed during key exchange phase. Peer verifies against stored public key.

Parameter	Default	Description
auth_mode	psk	psk = implicit via decryption. ed25519 = explicit signature verification.
auth_keydir	~/.config/mpcp/keys/	Keypair and trusted peer public key storage.

---

6. Phase 1 — Accept Mode

PC1 starts in accept mode. No ports opened. Raw socket monitors for calibration traffic. If NAT traversal enabled, contacts rendezvous server with SHA256(session_id || psk_bytes) first. Port timing thread launched immediately with SCHED_FIFO priority so it is ready before the first chunk arrives.

> **NOTE:** *Requires CAP_NET_RAW or root. Drop all other privileges after startup.*

---

7. Phase 2 — Calibration

7.1 Randomised Ping Count

Ping count drawn randomly from configured range each session:

```
ping_count = getrandom() mod (ping_count_max - ping_count_min) + ping_count_min
```

Parameter	Default	Description
ping_count_min	60	Minimum calibration pings
ping_count_max	140	Maximum calibration pings
batch_size	10	Pings per sub-batch in standard mode

7.2 Standard Mode / Slow Mode / Disguised Calibration

Standard mode: parallel burst (~1 RTT total). Slow mode: one ping per randomised interval over minutes. Disguised: packets formatted as DNS/NTP. All three are independently togglable. See Section 16 for profile combinations.

7.3 RTT Estimation Pipeline: Trimmed Mean + MAD + EWMA

> **v0.5:** *Replaces the single trimmed-mean estimator from v0.4. The three-stage pipeline produces a stable, continuously-updated RTT estimate that is robust to outliers, skewed distributions, and changing network conditions.*

The three stages are applied in sequence to the collected ping RTT samples:

Stage 1 — Trimmed Mean

Sort all N RTT samples. Discard the bottom trim_pct% and top trim_pct% (default 10% each). Compute the mean of the remaining 80%. This removes transient spikes and floor anomalies from the baseline estimate.

```
sorted = sort(rtts)
lo = floor(N * trim_pct / 100)
hi = N - lo
trimmed_mean = mean(sorted[lo:hi])
```

Stage 2 — Median Absolute Deviation (MAD)

MAD is a robust spread estimator that, unlike standard deviation, is not inflated by outliers or skewed distributions. It measures the median of the absolute deviations from the median.

```
median_rtt   = median(rtts)
deviations   = abs(rtt[i] - median_rtt)  for each i
MAD          = median(deviations)
robust_std   = 1.4826 * MAD   // scale factor for normal-equivalent sigma
```

The 1.4826 scale factor makes MAD equivalent to standard deviation for a normally distributed dataset, while remaining resistant when the distribution is skewed (as WiFi and internet RTT distributions often are). robust_std replaces flight_stddev in the catch window formula.

Stage 3 — EWMA (Exponentially Weighted Moving Average)

EWMA provides a continuously-updated flight time estimate that tracks current network conditions without discarding historical data entirely. It is applied during the pre-transfer mini re-calibration and updated with each new measurement throughout the session:

```
// Initialise from calibration phase:
ewma_rtt = trimmed_mean

// Update on each new RTT sample (mini re-cal and live):
ewma_rtt = (ewma_alpha * new_sample) + ((1 - ewma_alpha) * ewma_rtt)
```

Parameter	Default	Description
trim_pct	10	Percentage of samples trimmed from each end for trimmed mean (10 = trim bottom 10% and top 10%)
ewma_alpha	0.15	EWMA smoothing factor. Lower = more inertia, slower to track changes. Higher = more reactive.

The final catch window uses trimmed_mean as the base flight estimate and robust_std (MAD-derived) as the spread estimate, with EWMA providing the live-updated flight_avg that blends smoothly as conditions change:

```
flight_avg    = ewma_rtt / 2
flight_spread = robust_std / 2
catch_window  = min(flight_avg + (stddev_multiplier * flight_spread) +
floor_buffer, max_catch_window)
```

Parameter	Default	WiFi	Description
stddev_multiplier	4.0	5.0	Spread headroom multiplier (applied to MAD-derived robust_std)
floor_buffer	30ms	50ms	Minimum catch window buffer
max_catch_window	500ms	600ms	Hard cap
ewma_alpha	0.15	0.10	Lower alpha on WiFi for more smoothing of spiky RTT

7.4 Multi-Source Key Material

```
master_secret = HKDF-SHA256(
    salt   = session_nonce,
    ikm    = timing_entropy || psk_bytes || getrandom(32) ||
clock_gettime_nanos,
    info   = "mpcp-v0.5-master"
)
```

7.5 HKDF Keystream

```
keystream    = HKDF-Expand(session_key, length = total_chunks * 64)
chunk_key[i] = keystream[i*64     : i*64+32]
port_seed[i] = keystream[i*64+32  : i*64+64]
port[i]      = port_seed[i] mod port_range + port_base
```

---

8. Phase 3 — Blind Key Exchange

8.1 Parallel Candidate Key Transmission

PC1 generates N candidate keys (key_candidates, default 10). PC1 opens N derived ports simultaneously and transmits all N keys in parallel. PC2 must have all N keys received before proceeding.

8.2 Constant-Time Blind Selection — Precise Operation Sequence

> **v0.5:** *v0.4 described the constant-time requirement but left the exact timing of the fixed delay ambiguous. v0.5 specifies the precise operation order to eliminate any implementation ambiguity.*

PC2 executes the following operations in this exact order with no deviation:

1. Record T_recv: timestamp when last of N keys arrives.
2. Select index: selected_idx = getrandom_uint32() mod N. This is the only non-constant-time operation and it occurs before any key material is examined.
3. Copy selected key into selected_buf using a constant-time loop that reads all N keys regardless of selected_idx. This prevents memory access pattern leakage.
4. Build return array: copy all N keys into return_buf[] using constant-time operations (no early exit, no branch on index).
5. Fisher-Yates shuffle return_buf[] using constant-time swap (XOR swap on fixed-length buffers, no branch).
6. sodium_memzero(selected_buf_copy) — zero the working copy.
7. Wait until T_recv + key_exchange_delay has elapsed. Sleep the remainder if any steps completed early. This fixed-duration wait masks all timing variance from steps 2-6.
8. Open N-1 derived ports simultaneously. Transmit shuffled return_buf[] in parallel.

> **SECURITY:** *Steps 2-6 must complete within key_exchange_delay. If any step takes longer, the implementation is too slow for the configured delay. Increase key_exchange_delay or optimise the implementation.*

Parameter	Default	Description
key_candidates	10	Number of candidate keys. Higher = more observer ambiguity.
key_exchange_delay	50ms	Fixed wait after T_recv before returning keys. Must exceed steps 2-6 execution time.

8.3 Session Key Finalisation

```
session_key = HKDF-SHA256(salt=master_secret, ikm=selected_candidate,
info="mpcp-v0.5-session")
```

All N-1 returned candidates zeroed with sodium_memzero() immediately. Session key stored in sodium_malloc() locked memory.

---

9. Phase 4 — File Transfer

9.1 Compressibility Detection

```
sample        = first 65536 bytes of file
ratio         = len(zstd_compress(sample, level=1)) / len(sample)
if ratio > 0.95: skip_compression = true
```

9.2 Streaming Pipeline

```
[DISK] —> [Reader T1] —ring0—> [Compress T2] —ring1—> [Crypto T3] —ring2—>
[Sender T4] —> [NETWORK]
```

Four-stage lock-free SPSC ring buffer pipeline. Memory usage O(pipeline_depth x chunk_size) regardless of file size. Ghost chunks generated on-the-fly in Sender thread — never stored.

9.3 Chunk Sizing and Distributed Remainder

Compressed Size	Target Chunk Size	Approx. Chunks
< 1 MB	~100 KB	~10
1 MB – 10 MB	~100 KB	10 – 100
10 MB – 100 MB	~128 KB	~100
100 MB – 1 GB	~128 KB	~800
> 1 GB	~256 KB	scales

Remainder distributed: first (compressed_size mod N) chunks get floor(size/N)+1 bytes, rest get floor(size/N). No short final chunk.

9.4 Chunk Encryption

Each chunk padded to chunk_pad_size, encrypted with XChaCha20-Poly1305. AEAD additional data: seq_index(4B) || session_nonce(32B). Binds ciphertext to position for post-decryption index verification.

9.5 Ghost Chunks

Parameter	Default	Description
ghost_chunks_enabled	true	Enable ghost chunk generation
ghost_chunk_min	5	Minimum ghost chunks
ghost_chunk_max	20	Maximum ghost chunks

9.6 Configurable Pipeline Depth

pipeline_depth	Behaviour	Use case
1	Fully sequential. One port open at a	Maximum stealth (default)

	time.	
2-4	Small parallel burst per round.	Balanced
5-8	Larger parallel burst.	Maximum speed, trusted network

9.7 ACK — BLAKE2b Sequence Hash + Jitter

```
ack_payload = BLAKE2b(sequence_index, key=session_key, outlen=4)
```

ACK delayed by random interval in [0, ack_jitter_max] before sending. Jitter bound derived from session_key.

9.8 Retry with Fresh Port Derivation

```
port_retry[i][r] = HKDF(session_key, "retry"||i||r||session_nonce) mod
port_range + port_base
```

9.9 Zero-Copy I/O

MSG_ZEROCOPY (Linux 4.14+) for send. sendmmsg()/recvmmsg() for batched syscalls when pipeline_depth > 1. Fallback gracefully if kernel does not support.

---

10. Phase 5 — Reassembly

```
  [NETWORK] —> [Receiver T1] —ring0—> [Verifier T2] —ring1—> [Decryptor T3]
—ring2—> [Writer T4] —> [DISK]
```

9. Receive into ring buffer. Bitmask dedup — drop duplicate seq_index silently.
10. Verify AEAD tag. Verify seq_index from AD. Failed chunks queued for retry.
11. Identify ghost chunks via ghost_map(session_key). sodium_memzero and discard.
12. Decrypt. Strip padding using plaintext_len field. sodium_memzero chunk key immediately.
13. Stream write verified data to disk in sequence order. No wait for all chunks.
14. After all chunks: sodium_memzero session_key and master_secret.
15. If skip_compression: done. Else: zstd stream decompress.

---

11. Session Termination

No termination signal. All key material zeroed per-chunk during reassembly with final pass at completion. No port remains open. Session simply stops producing traffic — no FIN, no BYE, no observable boundary.

12. NAT Traversal

12.1 Hashed Rendezvous Token

```
rendezvous_token = SHA256(session_id || psk_bytes)
```

Server receives opaque token. Stateless. Discards all data after exchange.

12.2 Hole Punching Flow

```
PC1                        Rendezvous                      PC2
[ SHA256(id||psk) ] ──> [ match tokens ] <── [ SHA256(id||psk) ]
[ recv PC2 addr ]   <── [ exchange IPs ] ──> [ recv PC1 addr ]
                         [ discard state  ]
[ UDP punch ──────────────────────────────────────────── UDP punch ]
[ direct P2P established ]
```

Parameter	Default	Description
nat_mode	auto	auto / direct / force_nat
rendezvous_host	(none)	Rendezvous server hostname or IP
rendezvous_port	7777	Rendezvous server port
rendezvous_timeout	30s	Peer registration wait timeout
hole_punch_attempts	5	Simultaneous UDP punch attempts per direction

13. WiFi Considerations

The RTT pipeline (trimmed mean + MAD + EWMA) handles WiFi jitter automatically. Recommended adjustments for heavily congested WiFi — or use --wifi profile (see Section 16):

Parameter	Default	WiFi	Reason
stddev_multiplier	4.0	5.0	Extra headroom for WiFi jitter
floor_buffer	30ms	50ms	802.11 retransmission cushion
max_catch_window	500ms	600ms	Congested 2.4GHz headroom
ewma_alpha	0.15	0.10	More smoothing on spiky RTT
pipeline_depth	1	1	Keep sequential on high-jitter links

14. Memory Safety

All key material allocated via sodium_malloc() (mlock'd, never swapped). Per-chunk keys zeroed immediately after use. Session key and master secret zeroed at completion.

Parameter	Default	Description
memory_lock	true	Use sodium_malloc/free for key material.

---

15. Security Audit Summary

ID	Vulnerability	Severity	Fix
VULN-01	Insufficient ping timing entropy	Critical	4-source HKDF master secret
VULN-02	No endpoint authentication	Critical	PSK implicit + Ed25519 optional
VULN-03	Fixed behavioral fingerprint	High	Randomised ping count + slow mode
VULN-04	Rendezvous IP correlation	High	SHA256(session_id || psk) token
VULN-05	ACK timing side channel	High	Derived random ACK jitter
VULN-06	Chunk count volume correlation	High	Ghost chunks with random count
VULN-07	Port derivation oracle	Medium	Per-session nonce in all derivations
VULN-08	Retry pattern leakage	Medium	Retry uses fresh derived port
VULN-09	Key selection timing attack	Medium	Constant-time ops + precisely timed fixed delay
VULN-10	Duplicate chunk injection	Medium	Index in AEAD AD + bitmask dedup
VULN-11	PSK brute force	Low	Entropy enforcement + Diceware generator
VULN-12	Key material memory exposure	Low	sodium_mlock + per-chunk zero

---

16. Configuration System

16.1 Configuration File

```
# MPCP Configuration File v0.5  (~/.config/mpcp/mpcp.conf)

# PSK
# psk                   =
psk_min_entropy         = 40
psk_store               = false

# Authentication
auth_mode               = psk
auth_keydir             = ~/.config/mpcp/keys/

# Calibration
ping_count_min          = 60
ping_count_max          = 140
batch_size              = 10
slow_mode               = false
slow_mode_min_gap       = 2000
slow_mode_max_gap       = 8000
disguise_calibration    = false
disguise_protocol       = dns

# RTT estimation
trim_pct                = 10
ewma_alpha              = 0.15
stddev_multiplier       = 4.0
floor_buffer            = 30
max_catch_window        = 500

# Key exchange
key_candidates          = 10
key_exchange_delay      = 50

# Transfer
pipeline_depth          = 1
chunk_pad_size          = 131072
zstd_level              = 3
ghost_chunks_enabled    = true
ghost_chunk_min         = 5
ghost_chunk_max         = 20
ack_jitter_max          = 80
chunk_retry_timeout     = 2.0
decoy_encoding          = false

# Ports
port_base               = 10000
port_range              = 55000
```

```
# NAT
nat_mode                = auto
rendezvous_host         =
rendezvous_port         = 7777
rendezvous_timeout      = 30
hole_punch_attempts     = 5

# Memory and I/O
memory_lock             = true
zerocopy                = true
batch_syscalls          = true

# Tripwire
tripwire                = true
tripwire_z_threshold    = 3.5
tripwire_window         = 5
tripwire_chi_pvalue     = 0.05
canary_log              = ~/.config/mpcp/canary.log
```

16.2 Deterministic CLI Profiles

v0.5: *v0.4 profiles described effects informally. v0.5 defines each profile as an explicit, complete parameter set. When a profile flag is passed, exactly the listed parameters are set. All other parameters retain their config file or default values. Profile parameters cannot partially override each other — the last profile flag wins entirely.*

Profile: --default (implicit, no flag)

Parameter	Value
slow_mode	false
disguise_calibration	false
stddev_multiplier	4.0
floor_buffer	30
max_catch_window	500
ewma_alpha	0.15
pipeline_depth	1
ghost_chunk_min	5
ghost_chunk_max	20
tripwire_z_threshold	3.5
auth_mode	psk
decoy_encoding	false

Profile: --wifi

Parameter	Value	Reason
stddev_multiplier	5.0	Extra jitter headroom
floor_buffer	50	802.11 retransmission cushion
max_catch_window	600	Congested 2.4GHz
ewma_alpha	0.10	More smoothing on spiky RTT
pipeline_depth	1	Sequential safer on high-jitter
tripwire_z_threshold	4.5	Higher threshold — WiFi RTT naturally noisier

Profile: --fast

Parameter	Value	Reason
slow_mode	false	No calibration delay
floor_buffer	10	Tighter window on low-jitter LAN
stddev_multiplier	3.0	Less headroom needed
pipeline_depth	4	Parallel chunks for throughput
ghost_chunk_max	5	Fewer ghosts, faster transfer
ack_jitter_max	20	Less ACK delay
ewma_alpha	0.20	More reactive to conditions

Profile: --stealth

Parameter	Value	Reason
slow_mode	true	Spread calibration over minutes
slow_mode_min_gap	2000	Randomised inter-ping floor
slow_mode_max_gap	8000	Randomised inter-ping ceiling
disguise_calibration	true	Calibration looks like DNS/NTP
ghost_chunks_enabled	true	Obscure chunk count
ghost_chunk_min	10	More ghosts for better obscuring
ghost_chunk_max	30	More ghosts for better obscuring
pipeline_depth	1	One port open at a time

auth_mode	ed25519	Explicit endpoint verification
tripwire_z_threshold	3.0	More sensitive detection
ack_jitter_max	120	More ACK timing noise

Profile: --stealth --decoy (combined)

All --stealth parameters as above, plus:

Parameter	Value	Reason
decoy_encoding	true	Chunks formatted as C source

Profile: --internet

Parameter	Value	Reason
nat_mode	auto	Enable hole punching if needed
ewma_alpha	0.12	Moderate smoothing for variable internet RTT
stddev_multiplier	4.5	Extra headroom for internet jitter
max_catch_window	500	Keep cap reasonable
ack_jitter_max	120	More jitter noise on internet paths
tripwire_z_threshold	4.0	Internet paths naturally noisier

16.3 Profile Combination Rule

If multiple profile flags are passed (e.g. --wifi --stealth), the last flag on the command line wins entirely. Its full parameter set is applied, overriding any earlier profile. Individual --param overrides may be applied after a profile to tune specific values.

```
# Example: stealth profile but with faster ACKs
mpcp --stealth --ack-jitter-max 40 send file.tar peer_ip
```

---

17. Full Attack Surface Table

Attack Vector	MPCP v0.5 Defense
Port scanning	Ports open <200ms on HKDF-derived numbers from 55k range

Key interception	4-source HKDF; PSK + 256-bit OS random never transmitted
Entropy brute force	PSK (40-100+ bits) + 256-bit OS random; combined infeasible
Endpoint impersonation	PSK implicit auth; optional Ed25519
Replay attack	Per-session nonce; AEAD; seq index in AD; bitmask dedup
Partial capture	Individually keyed chunks + ghost chunks; any subset useless
Behavioral fingerprint	Randomised ping count; slow mode; no fixed sequence
File signature analysis	zstd + encryption + uniform padding = opaque uniform bytes
File size inference	Ghost chunks + distributed remainder obscure size
Key exchange interception	Parallel send; precise constant-time selection; fixed delay
Timing attack on key selection	Constant-time loop reads all N keys; fixed-duration wait
ACK timeline reconstruction	Per-chunk random ACK jitter derived from session key
Port derivation oracle	Per-session nonce; sequences unique across sessions
Retry pattern leakage	Retries use freshly derived ports with retry count
Rendezvous IP correlation	SHA256(id||psk) token; server sees nothing linkable
Memory scraping	sodium_mlock; per-chunk zero; no swap
DPI fingerprinting	Disguised calibration; no MPCP wire signature; decoy option
Duplicate chunk injection	Index in AEAD AD verified post-decrypt; bitmask dedup
PSK brute force	Entropy enforcement; Diceware generator; mlock prevents disk exposure
Active interception	Z-score tripwire + chi-squared loss; network-adaptive threshold; silent abort

18. Wire Format Specification

All packets sent over UDP. All multi-byte integers big-endian. All packets share a 6-byte common header: magic(4) + version(1) + type(1). Magic: 0x4D435043 ('MCPC' — deliberate mismatch to avoid naive string search). Version: 0x05.

18.1 Calibration Packet

Offset	Length	Field	Description
0	4	magic	0x4D435043

4	1	version	0x05
5	1	type	0x01 = ping, 0x02 = pong
6	2	seq	Ping sequence number
8	8	send_ts	Sender timestamp, nanoseconds (uint64)
16	16	nonce_hint	First 16B of session_nonce for session identification
32	32	padding	Random bytes

18.2 Key Exchange Packet

Offset	Length	Field	Description
0	4	magic	0x4D435043
4	1	version	0x05
5	1	type	0x03
6	1	key_index	Candidate key index (0 to key_candidates-1)
7	1	direction	0x00 = PC1→PC2, 0x01 = PC2→PC1
8	24	xchacha_nonce	XChaCha20-Poly1305 nonce
32	48	encrypted_key	32B candidate key + 16B Poly1305 tag
80	16	padding	Random bytes

18.3 Data / Ghost Chunk Packet

Offset	Length	Field	Description
0	4	magic	0x4D435043
4	1	version	0x05
5	1	type	0x04 = data chunk, 0x05 = ghost chunk
6	4	seq_index	Chunk sequence index (uint32)
10	1	retry_count	0 = first attempt, 1+ = retry
11	1	flags	Bit 0: skip_compression. Bit 1: last_chunk. Bits 2-7: reserved.
12	4	plaintext_len	Original length before padding (uint32)
16	24	xchacha_nonce	XChaCha20-Poly1305 nonce, unique per chunk
40	chunk_pad_size	ciphertext	Encrypted padded data + 16B Poly1305 tag

NOTE: *AEAD additional data: seq_index(4B) || session_nonce(32B). Binds ciphertext to position.*

18.4 ACK Packet

Offset	Length	Field	Description
0	4	magic	0x4D435043
4	1	version	0x05
5	1	type	0x06
6	4	ack_seq	Acknowledged sequence index (uint32)
10	4	ack_hash	BLAKE2b(seq_index, key=session_key, outlen=4)
14	18	padding	Random bytes

18.5 Tripwire Bounce Packet

Offset	Length	Field	Description
0	4	magic	0x4D435043
4	1	version	0x05
5	1	type	0x07
6	4	bounce_seq	Sequence index this bounce corresponds to
10	32	chunk_hash	BLAKE2b-256 of received chunk ciphertext
42	16	padding	Random bytes

---

19. Threading Model

Thread	Scheduling	Affinity	Responsibility
Main	SCHED_OTHER	Any	Startup, config, orchestration, shutdown
Port Timing	SCHED_FIFO pri 80	Pinned core	Port open/close at precise nanosecond times. Never preempted.
Reader	SCHED_OTHER	Any	Read file blocks into ring_buf_0

Compressor	SCHED_OTHER	Any	zstd stream compress into ring_buf_1
Crypto	SCHED_OTHER	Any	Pad, key derivation, encrypt into ring_buf_2
Sender	SCHED_OTHER	Any	Transmit via sendmmsg/MSG_ZEROCOPY. Generate ghost chunks.
ACK Handler	SCHED_OTHER	Any	Receive/match ACKs, signal retry
Tripwire	SCHED_OTHER	Any	Z-score oracle + loss pattern monitor. Trigger abort.

Parameter	Default	Description
ring_depth	4	Slots per inter-thread ring buffer
timing_core_id	0	CPU core to pin port timing thread
zerocopy	true	Enable MSG_ZEROCOPY if supported
batch_syscalls	true	Use sendmmsg/recvmmsg when pipeline_depth > 1

---

20. Performance Optimisation Summary

Optimisation	Impact	Section
Streaming 4-stage pipeline	O(chunk) memory; hides all latency stages	9.2
Parallel key exchange	N× faster handshake phase	8.1
Single HKDF keystream	2N HKDF calls replaced by 1	7.5
Trimmed mean + MAD + EWMA	Stable RTT on WiFi and internet paths	7.3
Compressibility detection	Skip compression on media/archive files	9.1
Distributed chunk remainder	No size-leaking short final chunk	9.3
On-the-fly ghost chunks	Ghost chunks never stored in RAM	9.5
Configurable pipeline depth	Up to 8× chunk parallelism on trusted links	9.6
BLAKE2b ACK matching	Correct ACK matching under reordering	9.7

MSG_ZEROCOPY	Eliminates one memcpy per chunk (Linux 4.14+)	9.9
sendmmsg / recvmmsg	Batch UDP I/O, single kernel call	9.9
SCHED_FIFO timing thread	Prevents OS preemption during port open/close	19
CPU affinity pinning	Eliminates OS migration jitter on timing thread	19
Lock-free ring buffers	Zero-contention inter-thread data flow	19

---

21. Tripwire and Interception Detection

21.1 Overview

The tripwire monitors for three anomaly classes during transfer. Any trigger causes immediate silent abort, local canary log entry, and zero network signal.

21.2 Anomaly Class 1: Z-Score Timing Oracle

> **v0.5:** *v0.4 used a fixed tripwire_timing_thresh = 0.15ms which is meaningless on internet paths where baseline jitter already exceeds this. v0.5 replaces it with a z-score threshold that normalises against the measured RTT distribution, making the tripwire equally sensitive on LAN (low jitter) and internet (high jitter) without manual tuning.*

During calibration, the RTT distribution's mean and standard deviation are recorded as the baseline. During transfer, the bounce-back oracle measures per-chunk RTT. Each measurement is converted to a z-score:

```
z = (oracle_rtt - baseline_mean) / baseline_std
```

A z-score measures how many standard deviations a sample is from the baseline mean. On any network, an RTT that is 3.5 standard deviations above baseline is equally anomalous whether the baseline is 0.3ms (LAN) or 45ms (internet). The threshold is network-adaptive without requiring any configuration.

Z-score	Probability of natural occurrence	Interpretation
< 2.0	~5% — common	Normal network variation
2.0 – 3.0	~0.3% — uncommon	Congestion or transient event

3.0 – 3.5	~0.1% — rare	Suspicious — begin watching
> 3.5	< 0.05% — very rare	Likely anomaly — increment counter
> 5.0	Extremely rare	Almost certainly artificial

```
z = (oracle_rtt - baseline_mean) / baseline_std
if z > tripwire_z_threshold: anomaly_count++
if anomaly_count > tripwire_window: ABORT
```

Parameter	Default	WiFi	Internet	Description
tripwire_z_threshold	3.5	4.5	4.0	Z-score above which a sample is counted as anomalous. Higher = less sensitive, fewer false positives on noisy links.
tripwire_window	5	5	7	Consecutive anomalous samples before abort.

NOTE: *The z-score approach requires the baseline RTT distribution to have been measured during calibration. If calibration sample count is below 30, the z-score estimate is unreliable. In this case MPCP falls back to a conservative fixed threshold of baseline_mean * 3.*

21.3 Anomaly Class 2: Loss Pattern Analysis

Natural packet loss is random and independent. Selective interception produces non-random loss patterns detectable by chi-squared test:

```
if chi_squared_pvalue(lost_chunk_indices) < tripwire_chi_pvalue: anomaly
```

Parameter	Default	Description
tripwire_chi_pvalue	0.05	Loss pattern p-value threshold. Below this = non-random loss detected.

21.4 Anomaly Class 3: Unexpected Decoy Response

Decoy pings sent to derived non-existent IP addresses during calibration. Any response or surrounding timing shift indicates ARP spoofing or network-level MITM.

21.5 Abort Procedure

```
// On any tripwire trigger:
sodium_memzero(session_key, 32)
sodium_memzero(master_secret, 32)
sodium_memzero(all_chunk_keys)
```

```
close_all_sockets()
write_canary_log(timestamp, anomaly_type, SHA256(session_nonce))
exit(0)   // Silent. No network signal.
```

Parameter	Default	Description
tripwire	true	Enable interception detection
canary_log	~/.config/mpcp/canary.log	Local canary log path

---

22. Decoy Encoding

TOGGLE: *DECOY_ENCODING — disabled by default. Enable with --decoy or decoy_encoding = true.*

Encrypted chunks formatted as syntactically valid C header files containing static data tables. Hex values of encrypted bytes laid out as C array initialisers. Compiles successfully, produces meaningless output. Social misdirection layer only — no cryptographic benefit since data is already XChaCha20 encrypted.

```
/* autogenerated -- do not edit */
#ifndef __TABLE_4A2F91BC_H
#define __TABLE_4A2F91BC_H
static const uint32_t __init_table[] = {
    0x4a2f91bc, 0x00fe3341, 0x9981aac0, 0x17ff2200,
};
#endif
```

NOTE: *Decoy encoding adds ~3x size overhead (hex representation). Disable on bandwidth-limited links.*

---

23. Known Remaining Limitations

- Active injection: Disrupts timing key derivation and causes session failure. Tripwire detects and aborts. Session restart with new nonce not yet specified.
- Symmetric NAT: UDP hole punching fails. TURN relay not specified.
- Global passive adversary: IP correlation at both endpoints is a fundamental limitation without an anonymising overlay.
- Multi-file sessions: Each transfer is an independent session.
- IPv6: Expected to work with minor adaptation. Not explicitly specified.
- Decoy encoding size overhead: ~3x. Not suitable for bandwidth-constrained links.
- Z-score tripwire requires N >= 30 calibration pings for reliable baseline. Sessions with ping_count_min below 30 will use conservative fallback threshold.

24. Related Work

- Port Knocking / fwknop (SPA): MPCP extends with timing calibration, multi-source entropy, streaming transfer.
- ZRTP: Key material from shared observable channel. MPCP strengthens with PSK and OS entropy.
- Physical Layer Security: RF channel as shared secret. MPCP applies at network timing layer.
- Oblivious Transfer: N-of-1 selection without revealing choice. MPCP blind key selection is a practical variant.
- WebRTC / ICE / STUN: NAT traversal via STUN. MPCP rendezvous is minimal analogue with hashed tokens.
- QUIC: Pipelined multiplexed UDP. MPCP pipeline depth draws from QUIC's multiplexing model.
- Diceware: Random passphrase from word list. MPCP PSK generator uses same approach.

25. Implementation and Testing Plan

25.1 Build Environment

Recommended: Linux 5.15+ (for full MSG_ZEROCOPY support). Required packages:

```
# Fedora / RHEL
dnf install libsodium-devel libzstd-devel libpcap-devel gcc make

# Debian / Ubuntu
apt install libsodium-dev libzstd-dev libpcap-dev gcc make
```

Verify kernel supports MSG_ZEROCOPY:

```
uname -r   # must be >= 4.14
grep ZEROCOPY /boot/config-$(uname -r)   # CONFIG_NET_SOCK_MSG=y
```

25.2 Linux Real-Time Thread Setup

The port timing thread requires SCHED_FIFO and optionally a dedicated isolated CPU core. Two steps:

Step 1 — Allow SCHED_FIFO without root (optional but recommended)

```
# /etc/security/limits.conf — add for your user:
youruser  hard  rtprio  80
```

```
youruser  soft  rtprio  80
```

Step 2 — Isolate a CPU core for the timing thread (optional, maximum precision)

```
# Add to kernel command line in /etc/default/grub:
GRUB_CMDLINE_LINUX="... isolcpus=3 nohz_full=3 rcu_nocbs=3"

# Regenerate grub config:
# Fedora:
grub2-mkconfig -o /boot/grub2/grub.cfg
# Debian:
update-grub

# Reboot, then verify core 3 is isolated:
cat /sys/devices/system/cpu/isolated   # should show: 3
```

Set timing_core_id = 3 (or whichever core was isolated) in mpcp.conf. The port timing thread will be pinned to this core via pthread_setaffinity_np() and scheduled SCHED_FIFO priority 80.

25.3 Unit Test Plan

Test ID	Component	What to verify	Pass condition
T-01	PSK entropy check	Feed passphrases of known entropy	Rejects below psk_min_entropy, accepts above
T-02	Calibration ping count	Run 100 sessions, record ping_count each	All values within [ping_count_min, ping_count_max], uniform distribution
T-03	RTT pipeline (MAD+EWMA)	Inject RTT samples with known distribution + outliers	trimmed_mean within 5% of true mean; MAD correctly ignores outliers; EWMA converges within 20 samples
T-04	Catch window formula	Vary stddev_multiplier and floor_buffer	catch_window = min(formula_result, max_catch_window) exactly
T-05	HKDF keystream	Derive keystream on both sides independently	chunk_key[i] and port[i] identical on both sides for all i
T-06	Parallel key exchange	Run exchange 1000 times	PC1 always identifies exactly the missing key; no timing correlation between selection and response time
T-07	Constant-time key selection	Profile PC2 response time across 10,000 runs with known selected index	Standard deviation of response time < 1 microsecond (masked by fixed delay)
T-08	Ghost chunk generation	Transfer file, capture all packets	Ghost chunks indistinguishable from data chunks by size, type byte, and ciphertext distribution

T-09	Distributed remainder	Transfer files of various sizes	All chunks within 1 byte of each other before padding
T-10	Bitmask dedup	Replay chunk N twice at PC1	PC1 accepts first, silently drops second; no double-processing
T-11	AEAD index binding	Swap two chunks in transit	PC1 rejects both — seq_index in AD does not match ciphertext position
T-12	Tripwire z-score	Inject artificial RTT delays of 1, 2, 3, 4, 5 stddevs	Abort triggers at correct z-threshold; no false positives below threshold
T-13	Tripwire chi-squared	Simulate selective loss of every 10th chunk	Chi-squared test flags non-random pattern; abort triggers
T-14	Silent abort	Trigger tripwire condition	Zero bytes sent after abort; canary log written; no network signal
T-15	Memory zeroing	Run session under valgrind with --track-origins	No key material readable after each chunk processed; none after session
T-16	Streaming memory usage	Transfer 10GB file, monitor RSS	Peak RSS never exceeds pipeline_depth * chunk_pad_size * 4 stages + overhead
T-17	Retry fresh port	Simulate chunk loss, observe retry	Retry uses different derived port from original; original port never reopened
T-18	CLI profile determinism	Apply each profile, dump config	Exact parameter set matches profile table in Section 16.2
T-19	Decoy encoding	Enable --decoy, capture chunks	All chunks parse as valid C. gcc compiles without errors.
T-20	End-to-end integrity	Transfer files of 1KB, 1MB, 100MB, 1GB	SHA256 of received file matches sender original in all cases

25.4 Integration Test Plan

Test ID	Scenario	Setup	Pass condition
I-01	LAN wired transfer	Two machines, gigabit ethernet	Transfer completes, integrity passes, no tripwire
I-02	LAN WiFi transfer	One WiFi, one wired	Transfer completes with --wifi profile, catch window wider than LAN
I-03	Both WiFi	Both machines on WiFi (2.4GHz)	Transfer completes; catch window auto-widens via MAD+EWMA
I-04	NAT traversal	Both machines behind separate NAT routers	Hole punch succeeds; transfer completes via direct P2P
I-05	Slow mode	Enable --slow; 60-140 pings spread over minutes	Transfer completes; no IDS alert on standard Suricata ruleset

I-06	Disguised calibration	Enable --disguise-cal	Wireshark shows DNS/NTP-formatted packets during calibration
I-07	Tripwire LAN	Introduce 5ms artificial delay to 10 consecutive chunks	Abort triggers within tripwire_window chunks; canary log written
I-08	Large file streaming	Transfer 50GB file	Peak memory stable throughout; no OOM; integrity passes
I-09	Concurrent transfers	Two simultaneous MPCP sessions on same machine	Sessions fully isolated; keys and ports do not collide
I-10	Calibration script validation	Run mpcp_calibrate.sh, then real transfer	Catch window from script within 10% of actual window used in transfer

25.5 Performance Benchmarks

Run on each target platform after implementation. Record and compare:

- Calibration time (standard mode, slow mode)
- Key exchange time (parallel)
- Transfer throughput at pipeline_depth 1, 2, 4, 8 (MB/s)
- CPU usage during transfer (per thread)
- Peak RSS during 1GB transfer
- Port timing jitter: standard deviation of actual port open time vs derived target time (target: < 500 microseconds on isolated core)

25.6 Recommended Implementation Order

For a developer starting from scratch, the recommended build order minimises integration risk:

16. crypto.c — libsodium wrappers, sodium_malloc, HKDF, XChaCha20, BLAKE2b. Unit test T-05.
17. config.c — Config file parser, CLI flags, PSK entropy check, profile loading. Unit test T-01, T-18.
18. calibrate.c — Ping burst, RTT collection, MAD+EWMA pipeline, catch window. Unit test T-02, T-03, T-04.
19. keygen.c + exchange.c — Candidate key generation, parallel send/receive, constant-time selection. Unit test T-06, T-07.
20. chunker.c — Compressibility check, streaming compress, split, distributed remainder, pad, encrypt. Unit test T-09, T-11.
21. sender.c + receiver.c — Pipeline threads, ring buffers, port timing thread (SCHED_FIFO), catch mechanism. Unit test T-08, T-10, T-15, T-16, T-17.
22. stitch.c — Ghost discard, decrypt, reorder, decompress. Unit test T-10, T-11.
23. tripwire.c — Z-score oracle, chi-squared loss, abort procedure, canary log. Unit test T-12, T-13, T-14.

24. nat.c — Rendezvous client, hole punching. Integration test I-04.
25. disguise.c — DNS/NTP packet formatting. Integration test I-06.
26. End-to-end integration tests I-01 through I-10.
27. Performance benchmarks (Section 25.5).

Appendix A — Complete Session Flow Diagram

```
PC2 (Sender)                                          PC1 (Receiver)
──────────────────────────────────────────────────────────────────────
[ OUT OF BAND: agree PSK; optionally exchange Ed25519 pubkeys ]

[NAT] SHA256(id||psk) —> rendezvous <— SHA256(id||psk) — direct P2P established

                                                     [ accept mode: passive
monitor ]
                                                     [ port timing thread:
SCHED_FIFO ]
[ send rand(60-140) calibration pings ]  ————————>
                                         <————————  [ record recv timestamps ]
[ collect RTTs ]
[ Stage 1: trimmed mean (trim 10% each end) ]
[ Stage 2: MAD → robust_std = 1.4826 * MAD ]
[ Stage 3: EWMA initialised from trimmed_mean ]
[ HKDF(timing||psk||getrandom||ts) = master_secret ]
[ 10 mini re-cal pings; EWMA updated ]
[ keystream = HKDF-Expand(session_key, N*64) ]      [ derive same keystream ]

                                         <————————  [ generate N candidate keys
]
                                                     [ open N ports
simultaneously ]
[ recv all N keys ]
[ T_recv recorded ]
[ Step 1: selected_idx = getrandom() mod N ]
[ Step 2: constant-time copy all N keys ]
[ Step 3: constant-time Fisher-Yates shuffle N-1 ]
[ Step 4: sodium_memzero working copy ]
[ Step 5: wait until T_recv + key_exchange_delay ]
[ Step 6: open N-1 ports; send N-1 keys parallel ] —>
                                                     [ recv N-1; find missing ]
                                                     [ session_key =
HKDF(master+candidate) ]
                                                     [ sodium_memzero N-1
candidates ]
[ sodium_memzero returned keys ]

[ sample 64KB; compressibility check ]
[ STREAMING PIPELINE ]
```

```
  Reader→ring0→Compressor→ring1→Crypto→ring2→Sender
  Tripwire thread monitors z-score oracle + loss chi-squared

  for each chunk i (data+ghost shuffled via ghost_map):
    port[i] from keystream                          [ open port[i] at T_open[i]
]
    send chunk via sendmmsg/MSG_ZEROCOPY  ———————>
                                                   [ catch; port closes ]
                                                   [ verify AEAD + seq index ]
                                                   [ sodium_memzero chunk key ]
                                                   [ bounce BLAKE2b hash ]
                                         <————————  [ ACK + rand jitter ]
    z = (oracle_rtt - baseline_mean) / baseline_std
    if z > tripwire_z_threshold: anomaly_count++
    if anomaly_count > tripwire_window:
      sodium_memzero all; write canary; exit(0)

[ all sent ]                                        [ STREAMING REASSEMBLY ]
                                                    [ ghost discard, decrypt,
write ]
                                                    [ zstd decompress if needed
]
                                                    [ file recovered ]
[ sodium_memzero all key material ]                 [ same ]
[ silent exit ]                                     [ silent exit ]
```

Appendix B — Cryptographic Primitive Summary

Operation	Algorithm	Library
Chunk encryption	XChaCha20-Poly1305 (AEAD)	libsodium
Key derivation	HKDF-SHA256 (extract + expand)	libsodium
Keystream generation	HKDF-Expand (single call)	libsodium
Random number generation	getrandom() / CSPRNG	OS kernel
Constant-time comparison	sodium_memcmp()	libsodium
Secure memory zeroing	sodium_memzero()	libsodium
Memory locking	sodium_malloc() / sodium_mlock()	libsodium
Endpoint authentication	Ed25519 (optional)	libsodium
Rendezvous token	SHA-256	libsodium

ACK matching	BLAKE2b (4-byte output)	libsodium
Bounce oracle hash	BLAKE2b-256	libsodium
Spread estimation	MAD (Median Absolute Deviation)	stdlib math
Live RTT tracking	EWMA	stdlib math
Tripwire timing	Z-score normalisation	stdlib math
Loss pattern analysis	Chi-squared test	stdlib math
File compression	zstd streaming (configurable level)	libzstd

Appendix C — Stealth Profile Quick Reference

Profile	Use Case	Key Differences from Default
(default)	LAN / trusted network	All defaults. Sequential. Full tripwire.
--wifi	WiFi networks	stddev_mult=5.0, floor=50, max_win=600, alpha=0.10, z_thresh=4.5
--fast	Dev / trusted LAN	pipeline=4, floor=10, stddev_mult=3.0, ghost_max=5, ack_jitter=20
--internet	Internet transfers	nat=auto, alpha=0.12, stddev_mult=4.5, ack_jitter=120, z_thresh=4.0
--stealth	Monitored networks	slow+disguise+ghost*2+ed25519+pipeline=1+z_thresh=3.0 +ack_jitter=120
--stealth --decoy	Max misdirection	All stealth + decoy encoding

---

Appendix D — Vulnerability Reference

ID	Vulnerability	Severity	Fix Section
VULN-01	Insufficient ping timing entropy	Critical	7.4
VULN-	No endpoint authentication	Critical	5

02			
VULN-03	Fixed behavioral fingerprint	High	7.1
VULN-04	Rendezvous server IP correlation	High	12.1
VULN-05	ACK timing side channel	High	9.7
VULN-06	Chunk count volume correlation	High	9.5
VULN-07	Port derivation oracle	Medium	7.5
VULN-08	Retry pattern leakage	Medium	9.8
VULN-09	Key selection timing attack	Medium	8.2
VULN-10	Duplicate chunk injection	Medium	10
VULN-11	PSK brute force	Low	5.2
VULN-12	Key material memory exposure	Low	14

Appendix E — Wire Format Quick Reference

Packet Type	Type Byte	Fixed Size	Key Fields
Calibration ping/pong	0x01 / 0x02	64B	seq(2), send_ts(8), nonce_hint(16)
Key exchange	0x03	96B	key_index(1), direction(1), xchacha_nonce(24), encrypted_key(48)
Data chunk	0x04	40B + chunk_pad_size	seq_index(4), flags(1), plaintext_len(4), nonce(24), ciphertext
Ghost chunk	0x05	40B + chunk_pad_size	Same layout as data; contents random
ACK	0x06	32B	ack_seq(4), ack_hash(4)
Bounce	0x07	58B	bounce_seq(4), chunk_hash(32)

NOTE: *Common header: magic(4) = 0x4D435043, version(1) = 0x05, type(1). All integers big-endian.*

Appendix F — Thread Architecture Diagram

```
SENDER (PC2)
──────────────────────────────────────────────────────────────
[DISK]—>[Reader T1]—ring0—>[Compress T2]—ring1—>[Crypto T3]—ring2—>[Sender
T4]—>[NETWORK]
                                                                  ^ ghost
chunks
                                                                    generated
here
[ACK Handler T5] <──────────────────────────────────────── [NETWORK]
[Tripwire T6]    z-score oracle + chi-squared loss monitor
[Port Timing T7] SCHED_FIFO pri 80, pinned core, mlock stack
[Main T0]        orchestration

RECEIVER (PC1)
──────────────────────────────────────────────────────────────
[NETWORK]—>[Receiver T1]—ring0—>[Verifier T2]—ring1—>[Decryptor
T3]—ring2—>[Writer T4]—>[DISK]
              |                    |
              bounce hash          ghost discard (ghost_map)
              to PC2 oracle        bitmask dedup
[Port Timing T5] SCHED_FIFO pri 80, pinned core
[Main T0]        orchestration

SCHEDULING
Port Timing : SCHED_FIFO priority 80, pthread_setaffinity_np to isolated core
All others  : SCHED_OTHER, any core
Optional    : isolcpus=N nohz_full=N rcu_nocbs=N in kernel cmdline
```

End of MPCP Protocol Specification v0.5 — Final Draft

This document is the complete implementation reference for the Multi-Port Catch Protocol.